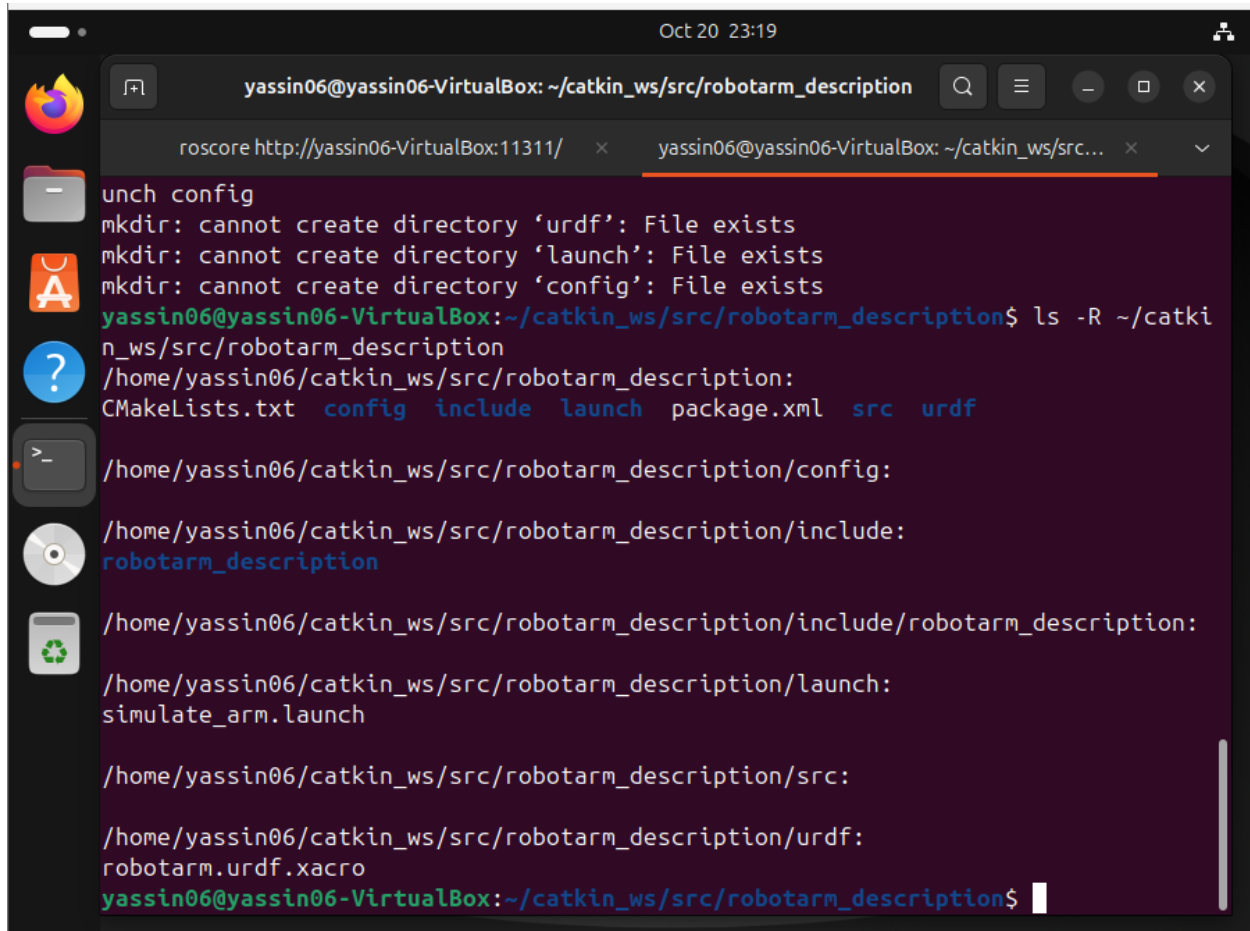


File Structure:

ls -R ~/catkin_ws/src/robotarm_description showing urdf/, launch/, config/, package.xml, CMakeLists.txt.



The screenshot shows a terminal window titled "yassin06@yassin06-VirtualBox: ~/catkin_ws/src/robotarm_description". The terminal output shows the following commands and results:

```
unch config
mkdir: cannot create directory 'urdf': File exists
mkdir: cannot create directory 'launch': File exists
mkdir: cannot create directory 'config': File exists
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description$ ls -R ~/catkin_ws/src/robotarm_description
/home/yassin06/catkin_ws/src/robotarm_description:
CMakeLists.txt  config  include  launch  package.xml  src  urdf

/home/yassin06/catkin_ws/src/robotarm_description/config:

/home/yassin06/catkin_ws/src/robotarm_description/include:
robotarm_description

/home/yassin06/catkin_ws/src/robotarm_description/include/robotarm_description:

/home/yassin06/catkin_ws/src/robotarm_description/launch:
simulate_arm.launch

/home/yassin06/catkin_ws/src/robotarm_description/src:

/home/yassin06/catkin_ws/src/robotarm_description/urdf:
robotarm.urdf.xacro
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description$
```

Q1: Package Structure

I created a package called robotarm_description with the standard ROS dependencies (rospy, roscpp, std_msgs, sensor_msgs, urdf, xacro). The package has three main folders:

- urdf/: This is where the robot model lives - basically the file that describes what the robot looks like and how it moves.
- launch/: Contains launch files that start up all the necessary nodes at once. Instead of manually launching each node, I can just run one launch file and everything starts up together.
- config/: Stores my RViz configuration so I don't have to reconfigure the display settings every time I open it.

Q2: Robot Model Description

My robot arm has 7 links total: a base_link at the bottom, then link1 through link5 for the arm segments, and finally tool0 at the tip. The links are connected by 6 revolute joints that rotate around different axes (Z, Y, Y, X, Y, Z).

For the geometry, I kept it simple - most of the arm segments are cylinders, and the end-effector is just a box. Each link has both visual and collision geometry so it shows up properly in RViz and can also be used for collision checking if needed.

Q3: Launch File Components

My simulate_arm.launch file handles all the assignment requirements:

Loading the robot model:

xml

```
<param name="robot_description" command="$(find xacro)/xacro '$(find robotarm_description)/urdf/robotarm.urdf.xacro'" />
```

This processes the xacro file and loads it as /robot_description.

Publishing transforms:

xml

```
<node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" />
```

This node takes the robot description and joint states to publish all the TF transforms.

Interactive control:

xml

```
<node name="joint_state_publisher_gui" pkg="joint_state_publisher_gui" type="joint_state_publisher_gui" />
```

This gives me sliders to control each joint manually.

Visualization:

xml

```
<node name="rviz" pkg="rviz" type="rviz" args="-d $(find  
robotarm_description)/config/robotarm.rviz" />
```

This opens RViz with my saved config that already has the robot model displayed.

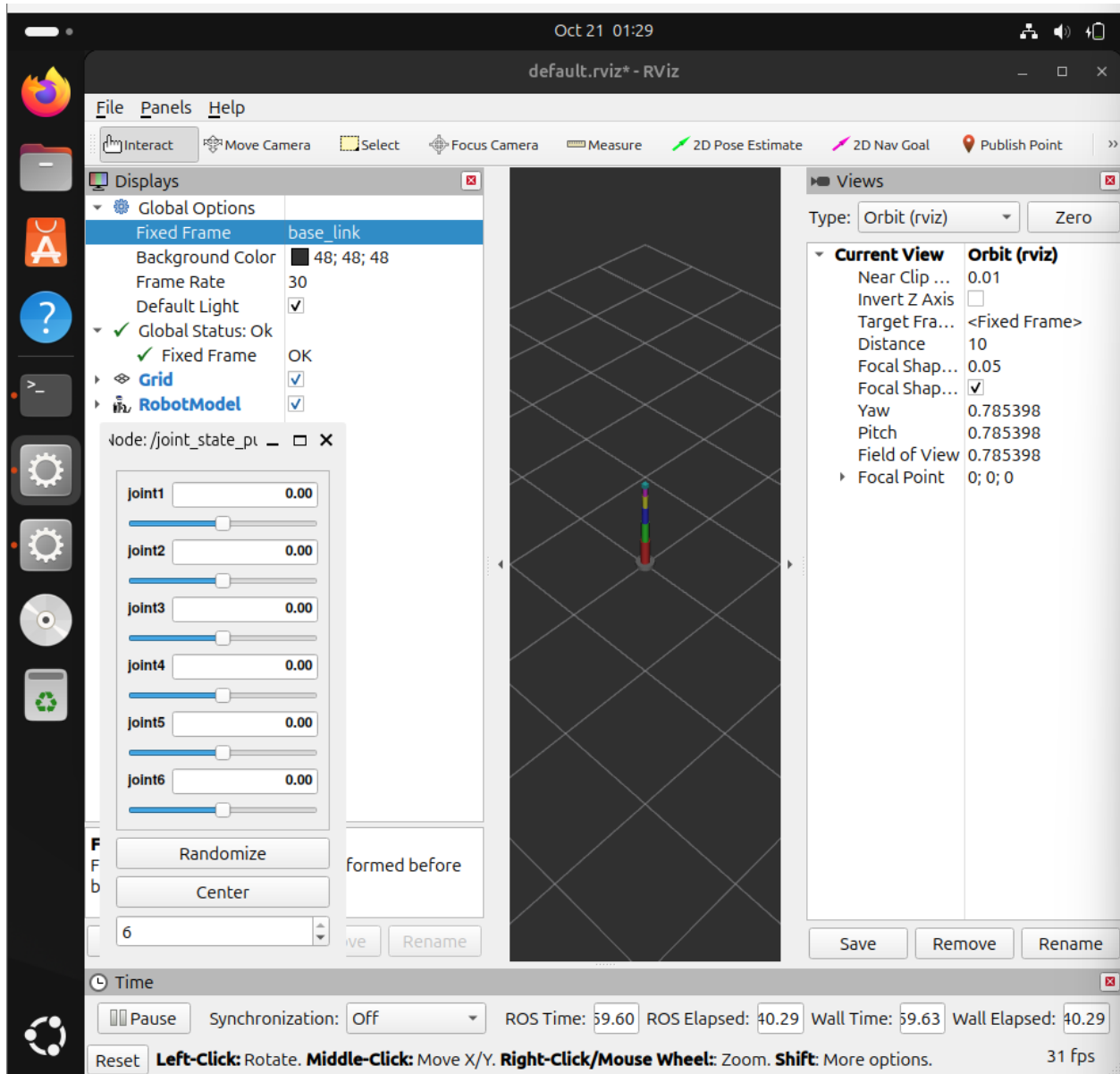
Q4: URDF Visual vs Collision Elements

The <visual> element is what I actually saw in RViz - it defines the colors, shapes, and appearance of each link. The <collision> element defines simplified geometry that's used for detecting contacts and collisions.

We need both because the visual model can be detailed and look nice, while the collision model should be simple (like basic shapes) to make collision calculations fast. For example, my visual links might be cylinders with specific dimensions, but the collision geometry could be even simpler approximations that are easier for the physics engine to work with.

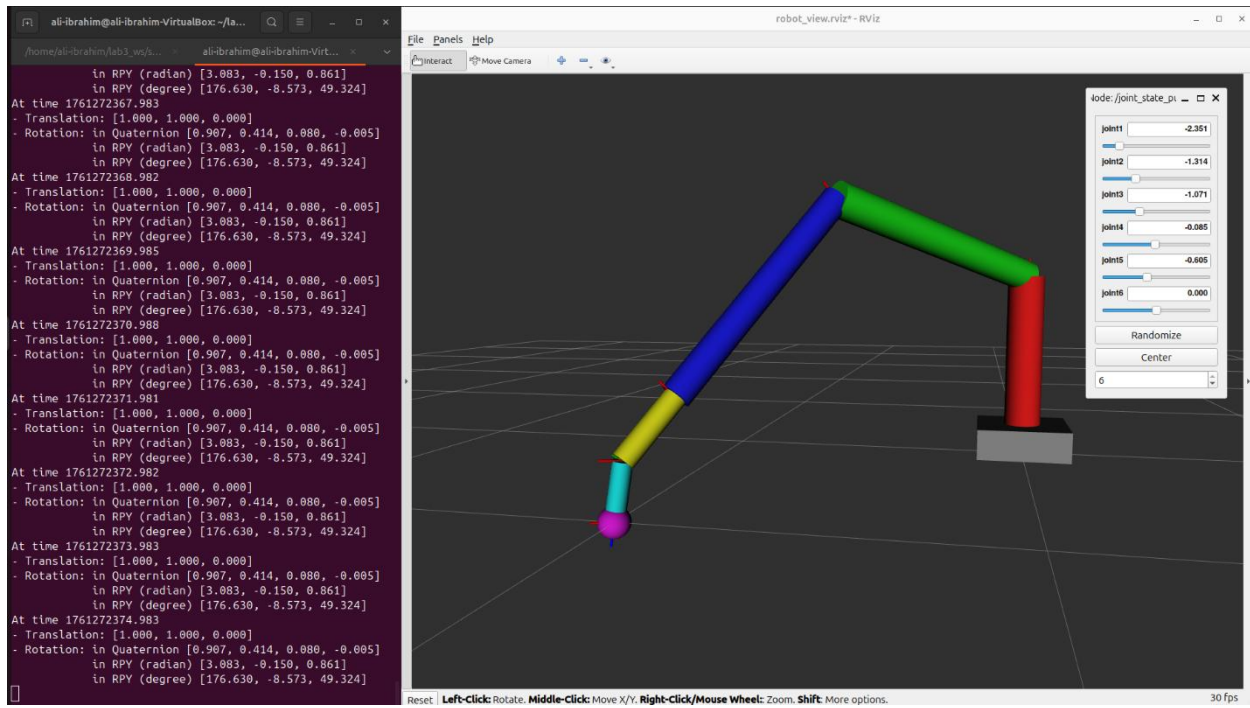
Default Pose:

RViz showing the robot in default pose + the slider window with values at 0.

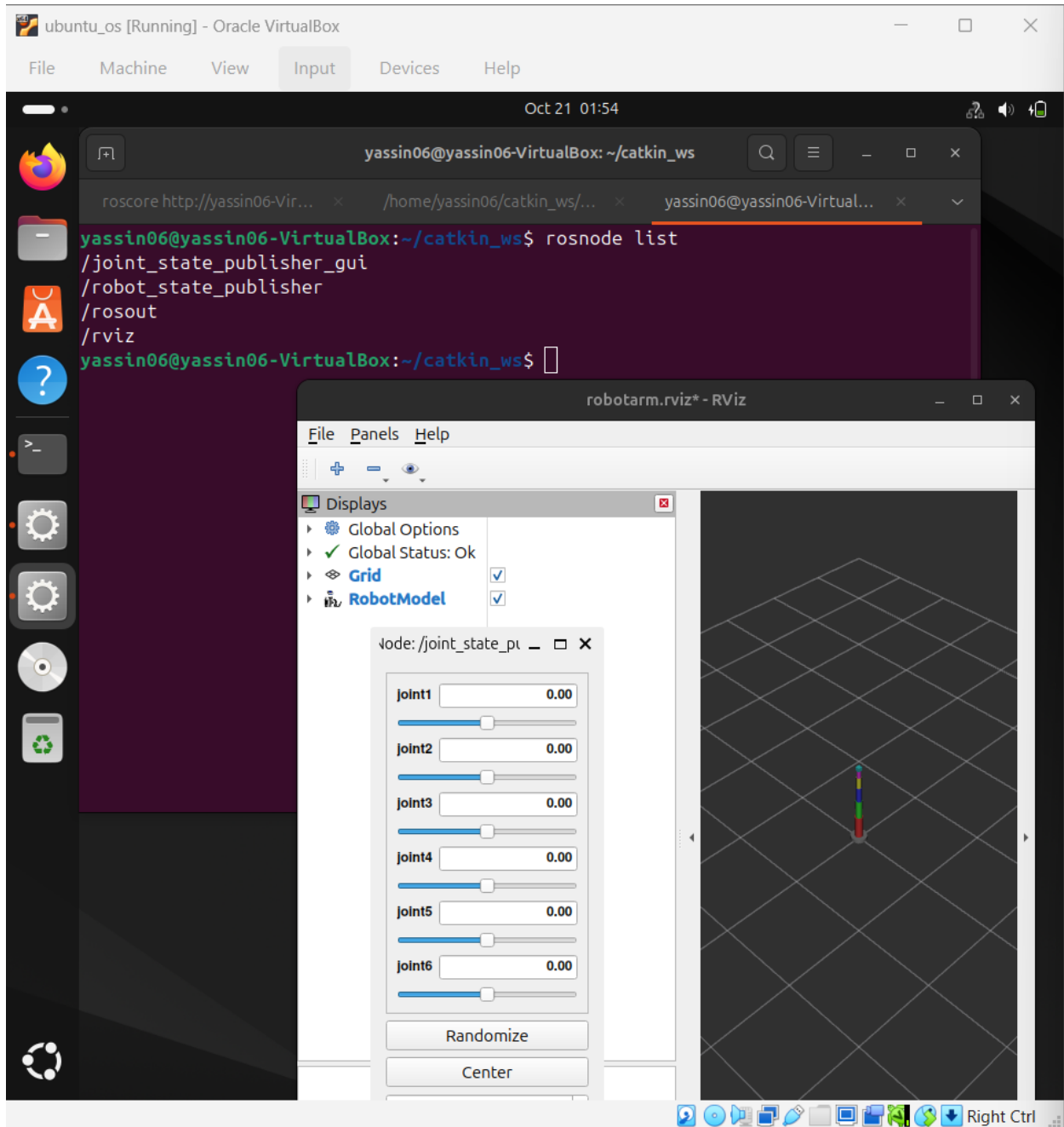


Pose 1 (1.0, 1.0, 0):

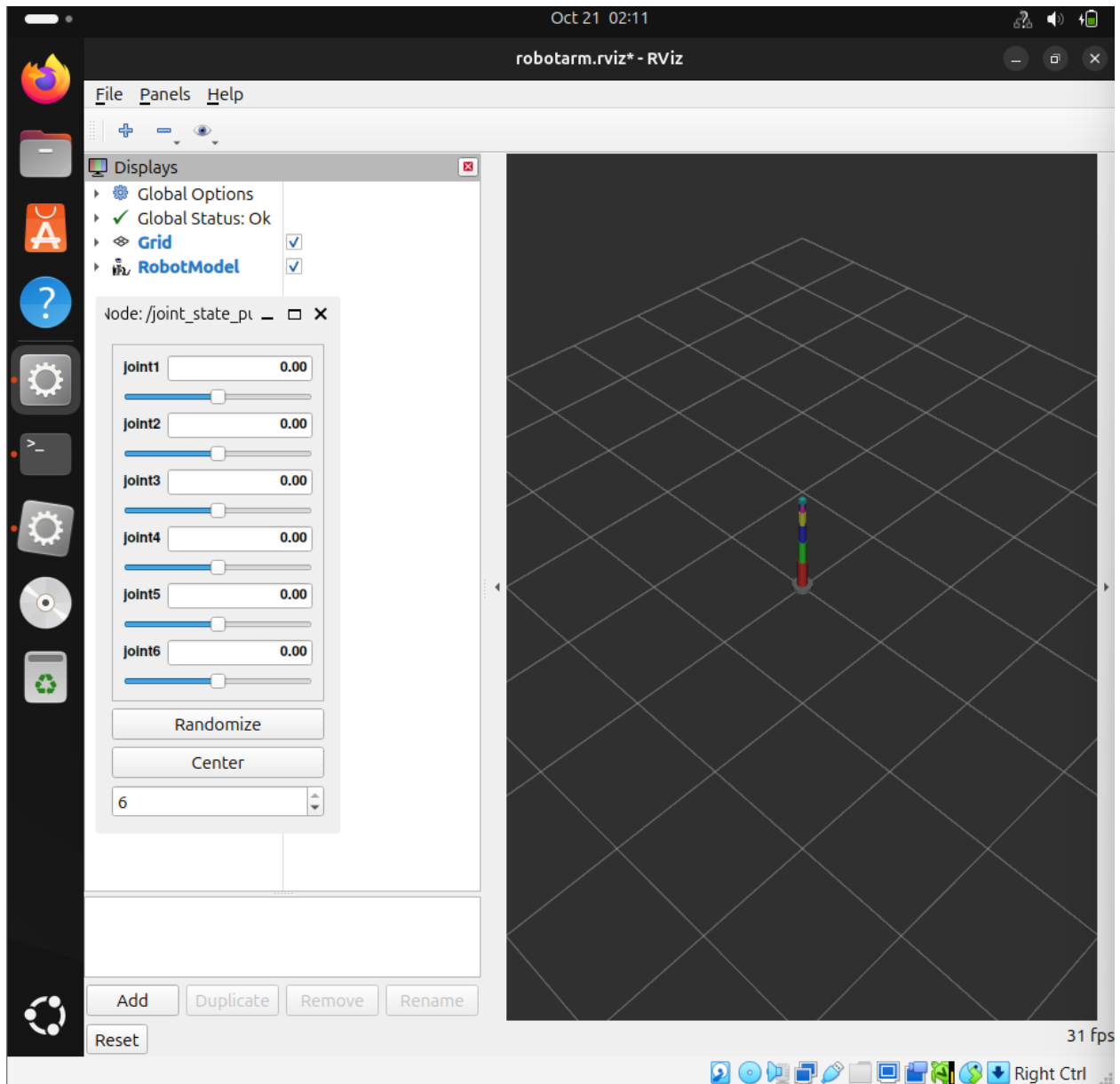
Robot posed at Pose 1



Screenshot 4



1.2:



Screenshot 2:

Show this terminal output (make sure the first few "Translation:" lines are visible).

This proves TF between base_link → tool0 works.

```
in RPY (degree) [104.066, -41.090, -100.603]
^Cyassin06@yassin06-VirtualBox:~/catkin_ws$ roslaunch tf tf_echo /base_link /tool0
At time 1761005833.779
- Translation: [-0.476, 0.228, 0.106]
- Rotation: in Quaternion [0.305, -0.706, -0.267, 0.581]
           in RPY (radian) [1.816, -0.717, -1.756]
           in RPY (degree) [104.066, -41.090, -100.603]
At time 1761005834.480
- Translation: [-0.476, 0.228, 0.106]
- Rotation: in Quaternion [0.305, -0.706, -0.267, 0.581]
           in RPY (radian) [1.816, -0.717, -1.756]
           in RPY (degree) [104.066, -41.090, -100.603]
At time 1761005835.478
- Translation: [-0.476, 0.228, 0.106]
- Rotation: in Quaternion [0.305, -0.706, -0.267, 0.581]
           in RPY (radian) [1.816, -0.717, -1.756]
           in RPY (degree) [104.066, -41.090, -100.603]
```

Screenshot for 5,

```
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ ls -lh
total 136K
-rw-rw-r-- 1 yassin06 yassin06 134K Oct 21 02:59 arm_motion.bag
```


Rosbag info:

```
yassin06@yassin06-VirtualBox: ~/catkin_ws/src/robotarm_description/bags
roscore http://ya... x /home/yassin06/c... x yassin06@yassin0... x yassin06@yassin0... x
bash: cd: baggs: No such file or directory
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description$ cd bags
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ rosbag r
ecord -O arm_motion.bag /joint_states
[INFO] [1761008276.893355234]: Subscribing to /joint_states
[INFO] [1761008276.903639753]: Recording to 'arm_motion.bag'.
^Cyassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ rosbag
info arm_motion.bag
Command 'r' not found, but can be installed with:
sudo apt install r-cran-littler
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ rosbag i
nfo arm_motion.bag
path:      arm_motion.bag
version:   2.0
duration:  1:05s (65s)
start:     Oct 21 2025 02:57:57.16 (1761008277.16)
end:       Oct 21 2025 02:59:02.26 (1761008342.26)
size:      134.0 KB
messages:  652
compression: none [1/1 chunks]
types:     sensor_msgs/JointState [3066dcd76a6cfaef579bd0f34173e9fd]
topics:    /joint_states 652 msgs : sensor_msgs/JointState
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$
```

Task 2:

2.1

Rosbag Topics:

Which topics are recorded in this rosbag and what message types do they use?"

```
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ rosbag info kurt.bag
path:          kurt.bag
version:       2.0
duration:      1:18s (78s)
start:         Nov 02 2012 14:50:47.67 (1351864247.67)
end:           Nov 02 2012 14:52:05.81 (1351864325.81)
size:          14.7 MB
messages:      28148
compression:   none [20/20 chunks]
types:         kurt_msgs/Encoder      [1b78a4027c07bd9a8a89543455494fb2]
               sensor_msgs/Imu       [6a62c6daae103f4ff57a132d6f95cec2]
               sensor_msgs/LaserScan [90c7ef2dc6895d81024acba2ac42f369]
               tf/tfMessage          [94810edda583a504dfda3829e70d7eec]
topics:        /encoder              7443 msgs      : kurt_msgs/Encoder
               /imu_recalibrated      7454 msgs      : sensor_msgs/Imu
               /scan                  5851 msgs      : sensor_msgs/LaserScan
               /tf                   7400 msgs      : tf/tfMessage
yassin06@yassin06-VirtualBox:~/catkin_ws/src/robotarm_description/bags$ rostopic list -b kurt.bag
/imu_recalibrated
/tf
/encoder
/scan
```

/encoder kurt_msgs/Encoder

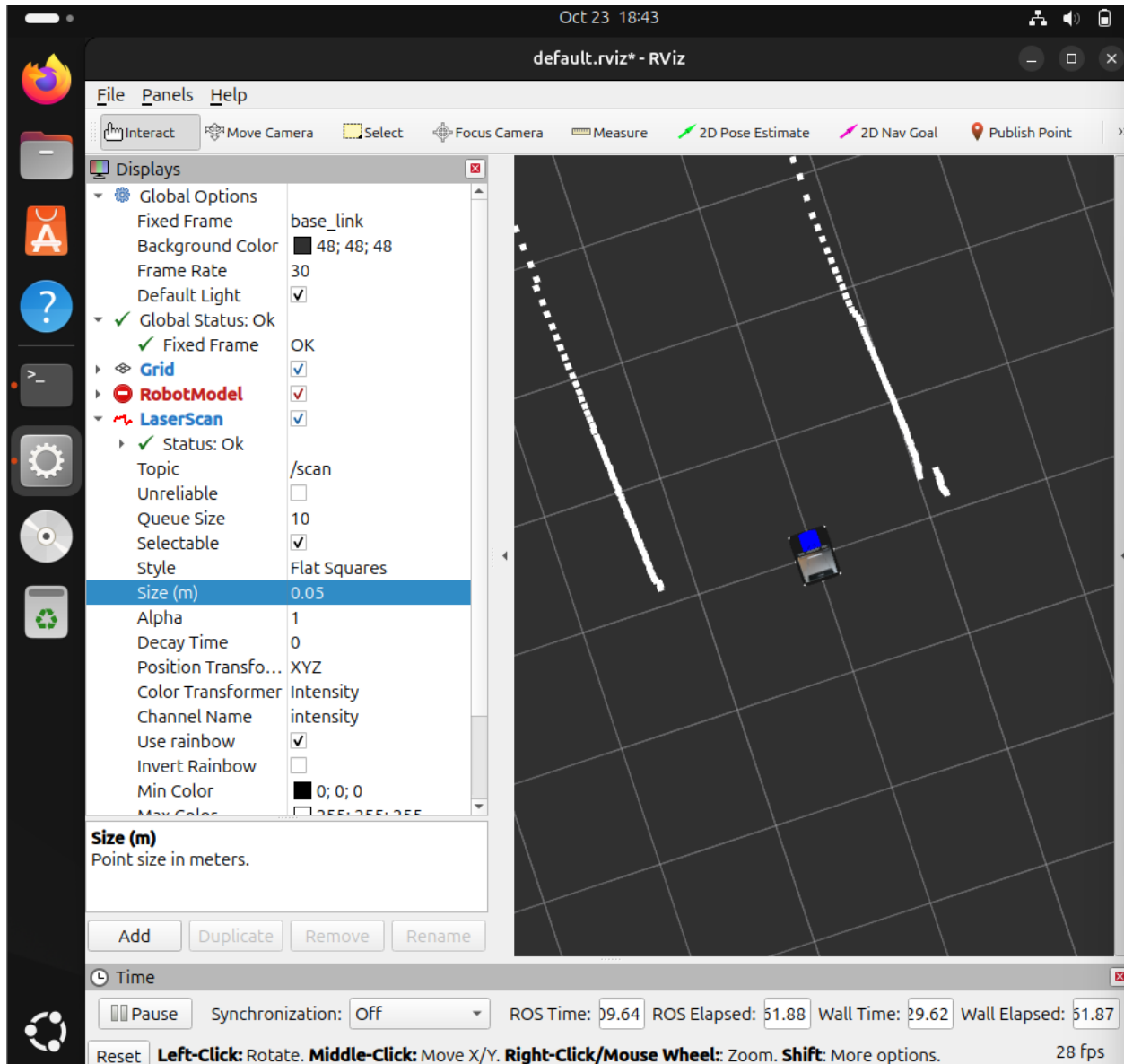
/imu_recalibrated sensor_msgs/Imu

/scan sensor_msgs/LaserScan

/tf tf/tfMessage

Screenshot C:

RViz with the RobotModel and LaserScan on display:



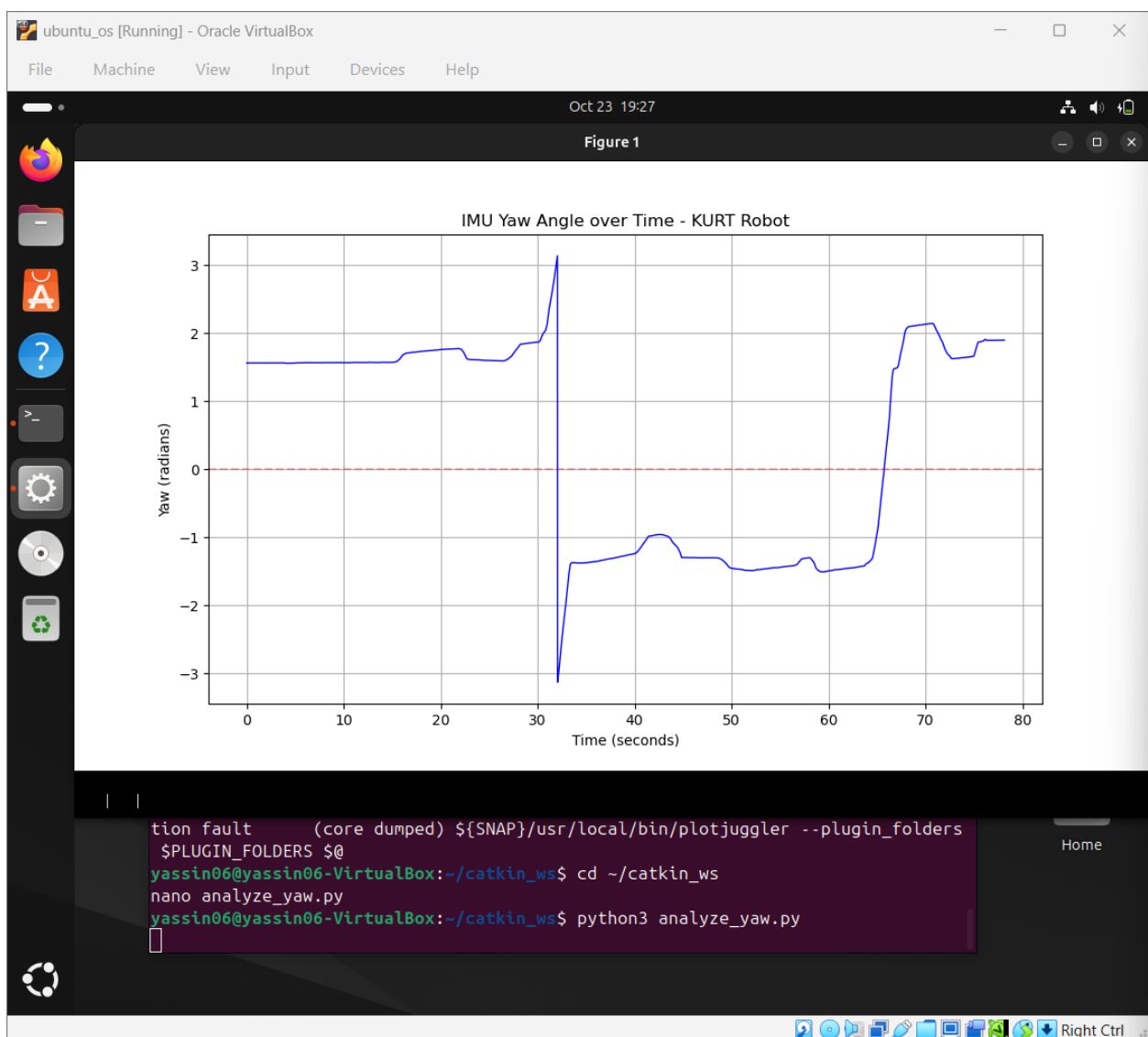
Q2 (RViz Visualization):

The visualization shows the KURT robot model with a 2D laser scan around it - you can see white dots forming a fan pattern that represents the laser readings. The scan shows the walls of an indoor corridor on both sides. As the bag plays through, you can see the scan pattern rotate and shift as the robot moves and turns down the hallway.

Q3 (What happened during recording):

The robot was driving through an indoor corridor with walls on both sides (you can see them clearly in the laser scan). It moved forward down the corridor, then did a turn-around, and came back along the same path. The laser scan rotating during the turn makes it pretty obvious when the robot changes direction.

Screenshot D:

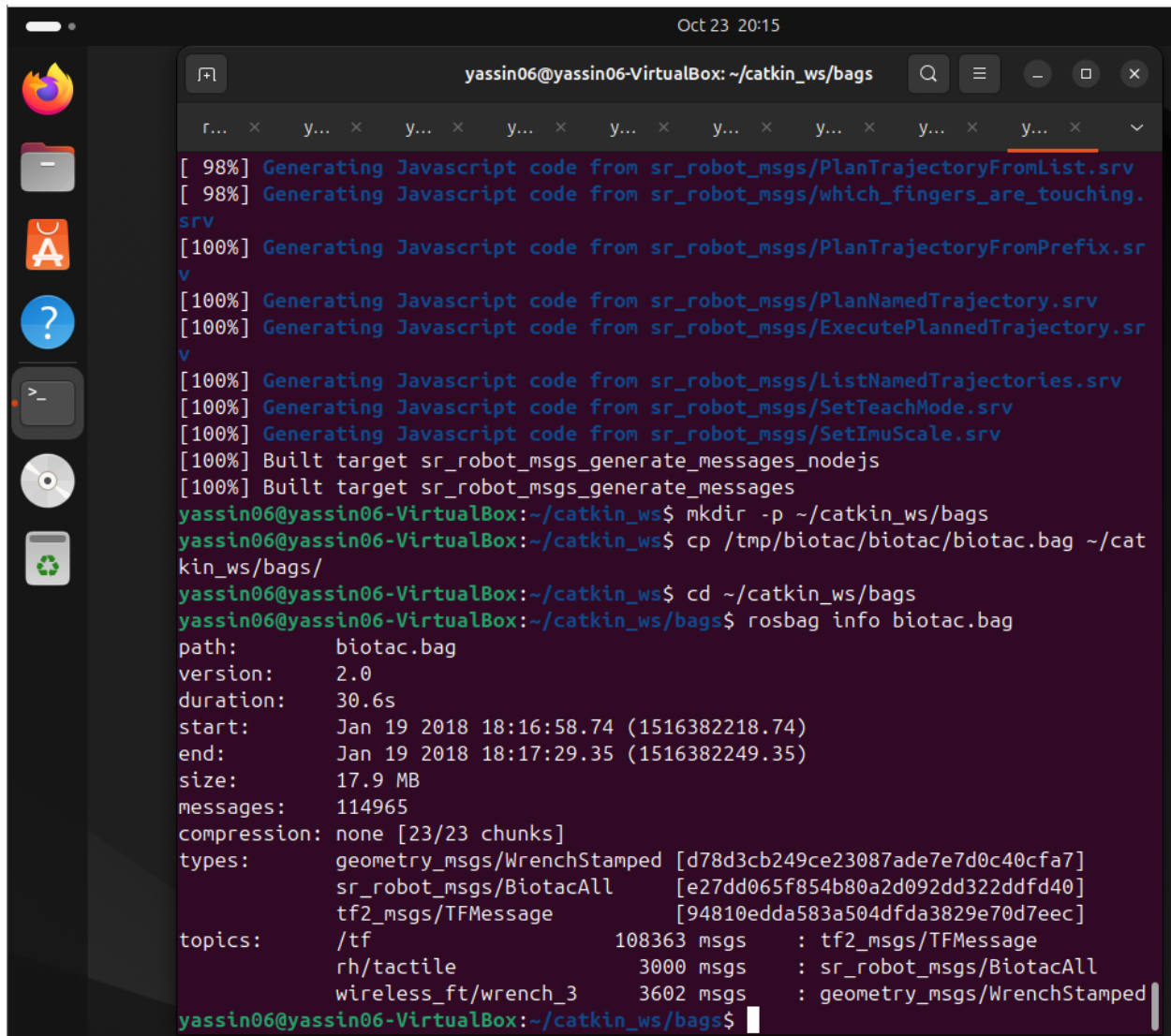


Q4 (Turn-around times from IMU yaw):

Looking at the yaw plot, there are two clear turn-arounds where the yaw angle jumps suddenly. The first one happens around 30 seconds, and the second one is around 68-70 seconds. These sharp jumps show when the robot did its 180-degree turns to reverse direction in the corridor.

Task 2.2:

Q5 (Topics & Types):



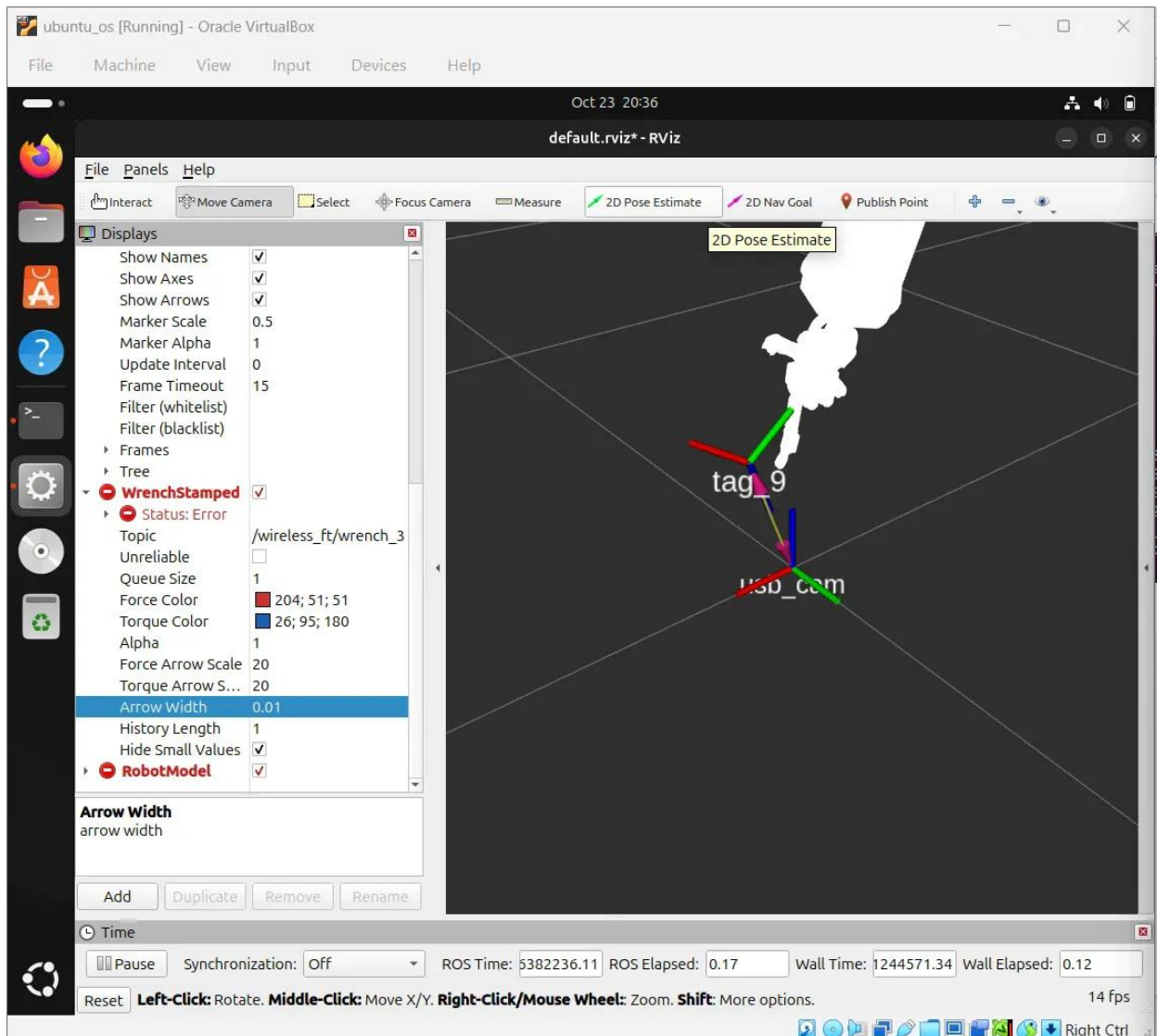
```
Oct 23 20:15
yassin06@yassin06-VirtualBox: ~/catkin_ws/bags

[ 98%] Generating Javascript code from sr_robot_msgs/PlanTrajectoryFromList.srv
[ 98%] Generating Javascript code from sr_robot_msgs/which_fingers_are_touching.srv
[100%] Generating Javascript code from sr_robot_msgs/PlanTrajectoryFromPrefix.srv
[100%] Generating Javascript code from sr_robot_msgs/PlanNamedTrajectory.srv
[100%] Generating Javascript code from sr_robot_msgs/ExecutePlannedTrajectory.srv
[100%] Generating Javascript code from sr_robot_msgs/ListNamedTrajectories.srv
[100%] Generating Javascript code from sr_robot_msgs/SetTeachMode.srv
[100%] Generating Javascript code from sr_robot_msgs/SetImuScale.srv
[100%] Built target sr_robot_msgs_generate_messages_nodejs
[100%] Built target sr_robot_msgs_generate_messages
yassin06@yassin06-VirtualBox:~/catkin_ws$ mkdir -p ~/catkin_ws/bags
yassin06@yassin06-VirtualBox:~/catkin_ws$ cp /tmp/biotac/biotac/biotac.bag ~/catkin_ws/bags/
yassin06@yassin06-VirtualBox:~/catkin_ws$ cd ~/catkin_ws/bags
yassin06@yassin06-VirtualBox:~/catkin_ws/bags$ rosbag info biotac.bag
path:      biotac.bag
version:   2.0
duration:  30.6s
start:     Jan 19 2018 18:16:58.74 (1516382218.74)
end:       Jan 19 2018 18:17:29.35 (1516382249.35)
size:      17.9 MB
messages:  114965
compression: none [23/23 chunks]
types:     geometry_msgs/WrenchStamped [d78d3cb249ce23087ade7e7d0c40cfa7]
           sr_robot_msgs/BiotacAll      [e27dd065f854b80a2d092dd322ddfd40]
           tf2_msgs/TFMessage           [94810edda583a504dfda3829e70d7eec]
topics:    /tf                          108363 msgs : tf2_msgs/TFMessage
           /rh/tactile                  3000 msgs : sr_robot_msgs/BiotacAll
           /wireless_ft/wrench_3        3602 msgs : geometry_msgs/WrenchStamped
yassin06@yassin06-VirtualBox:~/catkin_ws/bags$
```

```
/tf          tf2_msgs/TFMessage
/rh/tactile   sr_robot_msgs/BiotacAll
/wireless_ft/wrench_3 geometry_msgs/WrenchStamped
```

Biotac Visualization:

RViz showing RobotModel + TF frames + Wrench visualization:



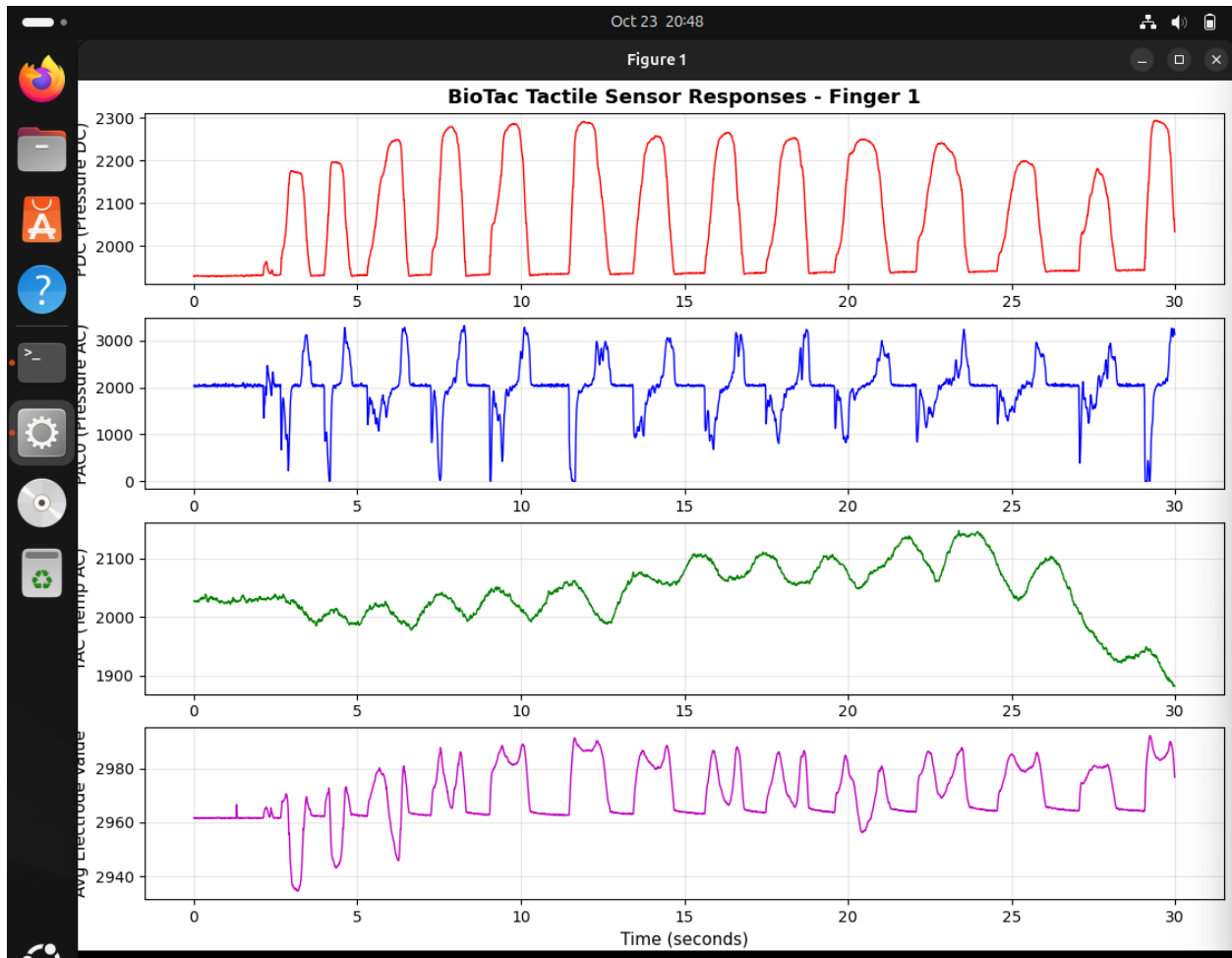
Q6 (RViz Visualization):

The visualization shows the BioTac robotic hand with TF coordinate frames visible - you can see the colored axes (red, green, blue for X, Y, Z) at different frame locations like tag_9 and usb_cam. There are also force and torque arrows coming from finger 3's sensor - the red arrow shows force and the blue arrow shows torque. These arrows indicate the forces that finger was experiencing during contact with objects.

Q7 (What happened during recording):

The recording captured data from the BioTac hand while one of the fingers was touching and manipulating something. You can see from the force/torque arrows that finger 3 was definitely in contact with objects - the red and blue arrows change in size throughout the recording, showing different amounts of force and torque. It looks like the hand was actively pressing, grasping, or manipulating objects for about 30 seconds based on how the sensor readings vary.

Screenshot G:



Q8 (Sensor fields showing response):

The PDC (Pressure DC) and PAC0 (Pressure AC) sensors show the clearest responses. PDC has really obvious square-wave patterns with peaks around 2200-2300, showing repeated contact events every 2-3 seconds. PAC0 has dramatic spikes that match up with when contact happens. The electrode values also show periodic changes that follow the same pattern. Based on all this, it looks like the finger was repeatedly tapping or pressing against something in a rhythmic pattern throughout the recording.

Contributions:

Task 1.1:

- Ali Ibrahim & Yassin Baher – Everything

Task 1.2:

- Ali Ibrahim – Everything

Task 2.1:

- Yassein Mahmoud & Yassin Baher – Everything

Task 2.2:

- Yassin Baher – Everything